IOP Publishing Mach. Learn.: Sci. Technol. 6 (2025) 045032 https://doi.org/10.1088/2632-2153/ae1563

MACHINE LEARNING
Science and Technology

OPEN ACCESS

RECEIVED
13 June 2025

REVISED
16 October 2025

ACCEPTED FOR PUBLICATION
20 October 2025

PUBLISHED
4 November 2025

PAPER

Human-in-the-loop reinforcement learning for data quality monitoring in particle physics experiments

Olivia Jullian Parra[1,2,*], Julián García Pardiñas[2,3], Lorenzo Del Pianta Pérez[2], Maximilian Janisch[4], Suzanne Klaver[5], Thomas Lehéricy[4] and Nicola Serra[1,2,6]

[1] Department of Mathematical Modeling and Machine Learning, University of Zürich, Winterthurerstrasse 190, Zürich 8057, Switzerland
[2] Department of Experimental Physics, European Organization for Nuclear Research (CERN), Geneva 23, 1211, Switzerland
[3] Laboratory for Nuclear Science, Massachusetts Institute of Technology (MIT), 77 Massachusetts Ave, Cambridge, MA 02139, United States of America
[4] Department of Mathematics, University of Zürich, Winterthurerstrasse 190, Zürich 8057, Switzerland
[5] Faculteit der Betawetenschappen, Vrije Universiteit Amsterdam, De Boolean 1105, Amsterdam 1081 HV, The Netherlands
[6] Department of Physics, University of Zürich, Winterthurerstrasse 190, Zürich 8057, Switzerland
[*] Author to whom any correspondence should be addressed.

E-mail: olivia.jullianparra@uzh.ch

Keywords: machine learning, reinforcement learning, RLHF, anomaly detection, data quality monitoring, high energy physics

Abstract

Ensuring data integrity via data quality monitoring (DQM) is critical in large-scale particle physics experiments. This traditionally relies on labor-intensive manual inspection or static machine learning models, which are adequate for stable operating conditions. However, the current era of major detector upgrades at the large hadron collider (LHC) challenges this paradigm. These upgrades are followed by prolonged commissioning periods with frequent and unpredictable changes in detector conditions, a regime for which static models are ill-suited. To address this, we reframe DQM as a dynamic decision-making problem and introduce a human-in-the-loop reinforcement learning (RLHL) framework. Our proximal policy optimization agent learns both to classify data and to strategically decide when to query human experts, optimizing the automation-oversight balance. On synthetic data, the system rapidly adapts to abrupt condition changes and successfully learns from noisy labels. In a simulated online regime, the agent minimizes human intervention by requesting it mainly when its uncertainty is high. A preliminary study on a real offline dataset from the LHCb experiment demonstrates that our synthetic approach is a reasonable proxy for real-world scenarios. The algorithm generalizes effectively with only superficial hyperparameter tuning, robustly identifying anomalies even when trained on augmented data. This work presents a scalable, adaptive solution for semi-autonomous DQM.

1. Introduction

Large particle physics experiments at facilities such as the large hadron collider (LHC) (Aad *et al* 2008, Aamodt *et al* 2008, Alves *et al* 2008, Chatrchyan *et al* 2008) collect immense volumes of data over extended periods to probe fundamental physics. While each unit of collected data is highly valuable, the hardware and software of these detectors can malfunction, compromising the data and rendering it unusable for scientific analysis. Consequently, data quality monitoring (DQM) plays a critical role in detecting anomalies, allowing the source of the problem to be fixed and the compromised data to be discarded (see, e.g. Adinolfi *et al* (2017a), Pol *et al* (2019c, 2022), Abadjiev *et al* (2024)).

DQM in particle physics typically takes place in two main stages. In the *online* stage, data are checked in near real-time (often every few minutes) so that operational issues can be promptly identified and resolved, prioritizing speed over exhaustive analysis. Once data collection finishes, the *offline* stage enables a comprehensive examination of the recorded dataset, allowing resource-intensive analyses and, ultimately, the removal of compromised data segments. Despite its central importance, this workflow

remains largely manual: non-expert humans (called shifters) typically review histograms or high-level data representations. Such a procedure is labor-intensive-on the order of hundreds of shifters per year for large collaborations-and susceptible to inaccuracies driven by individual judgment, fatigue, or other human factors.

To address this issue, several DQM projects have introduced machine learning techniques (see Pol *et al* (2022) for a recent review). Some approaches employ supervised methods, such as boosted decision trees (Adinolfi *et al* 2017a) or convolutional neural networks (Pol *et al* 2019c), while others adopt semi-supervised strategies using autoencoders (Deja 2019, Pol *et al* 2019a, Asres *et al* 2023, Abadjiev *et al* 2024) or variational autoencoders (Pol *et al* 2019b).

The many machine learning methods for DQM (Pol *et al* 2022) often rely on static models trained under fixed conditions. In reality, the 'nominal' detector behavior changes over time due to evolving beam parameters, operational configurations, and hardware or software modifications. This challenge becomes especially pressing as the LHC experiments undertake major detector upgrades for the high-luminosity LHC (HL-LHC) era (Zurbano Fernandez *et al* 2020), since the period following the commissioning of an upgraded detector typically involves many frequent-sometimes unforeseen-changes that can last for several months of data collection. Both human-driven guidelines and static ML methods often struggle to adapt quickly.

This work presents, for the first time, the application of RL techniques to DQM in particle physics experiments. RL-based approaches for anomaly detection exist across various domains (Arshad *et al* 2022) and excel at adapting to evolving systems. Within particle physics, RL has been employed mostly for accelerator control, with a notable recent example being the use of a PPO-based (Schulman *et al* 2017) agent for proton beam intensity control at the Mu2e experiment (Xu *et al* 2023). This and other recent applications (Hirlaender *et al* 2023, Su *et al* 2023) build on established work in the field (Hanten *et al* 2019, Hirlaender and Bruchon 2020, Kain *et al* 2020, Pang *et al* 2020, Kafkes and Schram 2021), where low sample efficiency remains a key bottleneck for continuous training in dynamic environments. This same challenge can arise in DQM, prompting our use of data-augmentation techniques, which have been widely adopted in various contexts to bolster deep learning models (Shorten and Khoshgoftaar 2019, Wen *et al* 2021).

Our work focuses on RLHL, in which human feedback is used in the reward function to better align the algorithm's behavior with human objectives. This is particularly effective when objectives are easier for humans to judge than to formally specify. A recent survey of RLHL applications, including control systems, robotics, large language models and recommender systems, can be found in Kaufmann *et al* (2023).

We propose a novel approach that redefines DQM as a reinforcement learning (RL) problem guided by human feedback (RLHF). Rather than focusing solely on identifying anomalies, our framework aims to maximize the yield of high-quality data collected in the long term while minimizing operational costs. By framing DQM decisions as a dynamic control task, we enable an RL agent to:

1. **Adapt to changing conditions:** as operation conditions change over time, the agent updates its policy without resorting to complete model retraining.
2. **Balance automation and human input:** the agent only requests human labels when classification uncertainty is high, reducing the load on shifters but preserving their oversight when it matters.
3. **Extend to corrective actions:** a continuous RL formulation takes temporal dynamics into account, allowing to optimally weigh data quality against operational costs in the long term. Though the focus here is anomaly detection, our approach could be extended to trigger more extensive controls such as software resets or parameter retuning. Each action has an associated cost, and a RL agent can learn to balance the total amount of good-quality data against these costs.

We develop a prototype based on a multi-agent proximal policy optimization (PPO) algorithm and data augmentation techniques, training and validating primarily on synthetic data. Our synthetic environment relies on generic functional shapes and bin-by-bin fluctuations while accounting for key real-world challenges: random label noise, frequent updates to operating conditions, and limited training data. The main advantage of using synthetic data is that the studies remain general and independent of a specific experiment. In practice, applying this approach to real experimental data should only require some basic input/output processing and model tuning. To demonstrate this, we also test the algorithm's

performance on a dataset from the LHCb experiment at the LHC (Alves *et al* 2008), restricted to the offline scenario[7]. In accordance with the LHCb Collaboration's External Data Access Policy[8], the dataset used in this study cannot be made fully public. Nevertheless, the study is motivated by the possibility of using the algorithm to improve DQM in the experiment, and some of the authors of this paper being members of the LHCb collaboration.

This paper is organized as follows: section 2.1 describes our proof-of-concept environment, and section 2.2 outlines the proposed RL-based solution. We then present detailed experiments for application in the offline (section 3.1) and online (section 3.2) regimes, noting that although online DQM ordinarily precedes offline in practice, we begin with offline as it represents a simpler case. Section 4 summarizes our findings, addresses limitations, and suggests future research directions. Finally, in section 4.3, we conclude.

2. Methods

Section 2.1 describes the setup in which the DQM occurs, including different operational regimes. We then present our proposed algorithm in section 2.2.

2.1. Experimental setup

A machine starts in a nominal mode of operation, and is subject to random failures which shift it to one of several anomalous modes of operation. The machine produces a summary statistic of its mode of operation at regular intervals—a *data point*, in this paper a one-dimensional histogram represented by a fixed-length vector of real values. This constitutes a simplification of a typical real application, where each data point is commonly represented by many different histograms. For each data point, we may have access to the machine's mode of operation via a label provided by a shifter, indicating whether the mode of operation is nominal or anomalous. Depending on the experiment, the labels can reflect the ground truth or include some level of shifter-related noise. The experiments are performed using synthetic data, generated as described in appendix A.1.1. In addition, section 3.1.5 presents a study based on real-world data.

We distinguish two training regimes, *online* and *offline*. The labels will always be available in the offline case, and in the online case, only when requested by the algorithm. The transition between the modes of operation of the machine, which constitutes a Markovian process, is different between the offline and online regimes, and it will be specified in the following sub-sections. The RL point of view is elucidated further in section 2.2.1. The data points follow the same order as they would in real-world collection. This allows for studying the capability of our RL algorithm to adapt to changing conditions, especially abrupt changes to the distribution of anomalous and nominal data points.

All the experiments conducted in this study, as well as the framework created for the offline real-world application can be found in Olivia Jullian Parra (2025).

2.1.1. Offline regime

In the offline regime, analysis is performed on a fixed, chronologically ordered dataset of histograms. In this regime, each data point is treated independently, with a fixed probability of being anomalous ($p_{\text{anom}} = 0.3$ in our synthetic studies, used as a prudent proof-of-concept probability, although real-world estimates are typically lower, e.g. 14% in the LHCb dataset). The setup allows underlying distributions to evolve over the time, testing the algorithm's ability to adapt to changing conditions. Additional details on the offline regime, including label noise and synthetic data generation, are provided in appendix A.1.4.

2.1.2. Online regime

In this regime, the data is collected while the algorithm is running. This regime introduces an additional level of complexity with respect to the offline one: the need to repair the machine. Namely, if an anomaly is spotted (either by the shifter or by the algorithm), the machine is restarted and returned to its nominal mode of operation. Otherwise, the mode of operation of the machine remains unchanged. In both cases, the machine then evolves to a new mode of operation: if it is in an anomalous mode of operation, its mode of operation remains the same, and if the machine is in the nominal mode of operation, the next mode of operation will be anomalous with probability p_{anom} (set to the same value as in

[7] Restricting to the offline scenario is guided by practical constraints: testing online performance would demand a full deployment in the central software of the experiment and the usage of the algorithm during real operation, with consequences for data taking. The results shown here can help motivate such future studies.

[8] http://opendata.cern.ch/record/410.

the offline regime) or nominal with probability $1 - p_{\text{anom}}$. A detailed explanation of this setup can be found in appendix A.1.4.

As a second additional level of complexity, the shifter's labels used for the training are not always accessible, to emulate the time limitations experienced by the shifters working in the online regime. This will be discussed in detail in section 2.2.

2.1.3. Computing and software resources

The experiments described in this paper have been performed in a commercial MacBook Pro laptop, with an Apple M1 Pro chip, 8 CPU cores and 16 GB of RAM. All the experiments are lightweight, presenting minimal resources in terms of computing time (order of few minutes per experiment, except for the one described in section 3.1.4, that takes about 4 h, and the one in section 3.1.5, that takes about half an hour). The memory consumption is also minimal, with only the study on real data in section 3.1.5 requiring few hundreds of MB. The algorithm is written in PyTorch (Paszke *et al* 2019), adapting the PPO implementation in Barhate (2021). The Gym library (Brockman *et al* 2016) is used to create the RL environment.

2.2. RL algorithm

In this section, we introduce the PPO algorithm used in our studies. A comprehensive summary of all hyperparameters and their chosen values is provided in appendix A.5.

We propose a multi-agent PPO algorithm with two agents: a *predictor*, whose task is to assign a label (nominal / anomalous) to each histogram; and a *checker*, whose task is to decide, based on the histogram and on the decision of the predictor, whether to call a human shifter who will themselves provide a label. Readers unfamiliar with PPO will find a conceptual summary of the approach in appendix A.4.

We describe the environment in section 2.2.1, the actions and value functions of the actors in section 2.2.2, the training episodes in section 2.2.3, the rewards in section 2.2.4, the loss function optimized by the PPO approach in section 2.2.5, and finally the training details in section 2.2.6.

2.2.1. Environment

At a given time t, following the description in section 2.1, the environment state consists of the current data point and of the current mode of operation (nominal / anomalous) of the machine. If a shifter is called or if we are in the offline regime, the state may be augmented by a human label. Only the data point is available to the predictor and to the checker.

2.2.2. Agents and actions

There are two agents in our approach: the *predictor* and the *checker*. The predictor is in charge of deciding whether the mode of operation in a given state is nominal or anomalous (recall that only the data point is available to the predictor). Which decision to take defines the two actions of the predictor. The checker has to examine the predictor's response for a given data point and decide whether to obtain a label from an external shifter or not. If called by the checker, a shifter will provide a label that can be used to train the predictor. If not called by the checker, in the online regime, there is a fixed probability p with which a shifter will provide a label anyway. This fixed probability of providing a shifter's label is called random check. In the offline case, a label will always be provided, meaning that the checker's actions have no effect on the state evolution and can thus be discarded. Details about the agents' configuration can be found in appendix A.1.4. Since we are using PPO, we need to train, for each agent, two networks simultaneously (conceptual details can be found in appendix A.4.3): the *actor*, defining the policy of the agent, and the *critic*, approximating the value function of the decision process.

2.2.3. Training episodes

In the offline regime, each individual histogram constitutes one training episode. In the online regime, all histograms between one random check, as described in appendix 2.2.2, and the next one, define the training episode. This is independent of the actions of the checker (if the checker requests a check, an episode divide happens if and only if a random check would have occurred without the request of the checker). It should be noted that this definition of episode is merely a design choice and is not expected to significantly impact the results compared to other possible approaches to the same problem.

2.2.4. Rewards

In this section, we describe the rewards for the predictor and checker agents. A conceptual overview of RL rewards can be found in appendix A.4.1.

Predictor If the shifter's label is available in the current state, the reward is $+1$ if the predictor's decision matches the human label, and -1 otherwise. If the shifter's label is not available, the predictor's reward is zero.

Checker If the checker did not request a check, the reward is zero. If the checker requested a check, the reward is $\omega - p$, where ω is for the predictor's *mis-tagging 'probability'* (defined below) and $p \in (0,1)$ is a hyperparameter (we set it to 0.1) that regulates the amount of penalization given to the checker for calling the shifter 'unnecessarily', i.e. when the predictor is doing well. The ω variable is a proxy of the probability that the predictor outputs the wrong label, using its own output. It is constructed using the two logits $(lp_{\mathrm{n}}, lp_{\mathrm{a}})$ in the predictor's output, for nominal and anomalous predictions respectively. We compute 'probabilities' by passing this vector through a softmax layer, then define ω as the 'probability' associated to the outcome that was not chosen by the shifter, i.e. $\omega = p_{\mathrm{a}}$ if the shifter's label is 'nominal', and $\omega = p_{\mathrm{n}}$ otherwise. The checker is thus positively rewarded if it calls the shifter when the predictor's mis-tagging 'probability' for a given state is larger than p, and negatively rewarded otherwise. It should be noted that, with this reward scheme, the checker could in principle adopt a policy in which it never asks for a check. Since all rewards would then be 0, this policy would be stable. However, this collapse is prevented by the continuous exploration induced by the entropy term included in the PPO loss, as described in section 2.2.5.

2.2.5. Losses

The loss L we use is a modification of the standard clipped PPO loss L^{clip} described in appendix A.4.3. For a given agent, it consists of three terms, as in (equation (9)) (Schulman *et al* 2017), $L = L^{\mathrm{clip}} - c_1 L^{\mathrm{value}} + c_2 L^{\mathrm{entropy}}$, where the hyper-parameters c_1 and c_2 are non-negative real numbers. We use $c_1 = \frac{1}{2}$ and $c_2 = 0.5$ unless otherwise stated. The first term is simply the clipped loss used to update the parameters of the actor. The L^{value} loss is the loss for the critics, equal to the squared difference between the prediction of the value network and the observed reward, $L^{\mathrm{value}} = (V^t_\phi(S_t) - R_t)^2$, see appendix A.4.3 for details. Finally, L^{entropy} is the entropy of the policy on the current data point, in our case the cross-entropy loss $-\ln \pi_t(a_t|S_t)$. This term is added to encourage exploration that may lead out of local minima.

2.2.6. Network update

The procedure to update the actor and critic networks (see appendix A.4.3) makes use of the Adam optimizer (Kingma and Ba 2014) with a learning rate of 10^{-4}. A sequence of ten update steps is done for each state, unless otherwise stated.

2.2.7. Training and testing scheme

The training of the algorithms in the different experiments will be done following the order of the sequence of data points, training on a single data point at a time and over the whole dataset in the experiment. Both the accuracy of the algorithm (i.e. that of the predictor against the ground truth labels), the F1-score (i.e. the harmonic mean of precision and recall of the predictor against the ground truth labels), the Brier score (i.e. the predictor's accuracy of probabilistic predictions) and the cumulative reward of the RL policy are evaluated on the 'future' episode immediately after the current one. Unless explicitly stated, all the plots presented in the following sections use this method to compute accuracy.

To evaluate how the performance depends on the specific training trajectory, each experiment is repeated a certain number of times (10, unless otherwise specified). Each repetition uses the same set up for both training and testing data, but a different seed for the random number generator used in the generation of the states and in the initialization of the weights. As a result, ten experiments with the same set up and goals are preformed and thus, the plots shown in the following sections contain solid lines and error bands representing, respectively, the average and standard deviation over experiments. A binning of 20 episodes is used when computing those statistics in all the plots except for the ones in section 3.1.4, where they are computed for single episodes.

3. Results

3.1. Experiments in the offline regime

In the following subsections, we study the algorithm's capacity to adapt to changing conditions (section 3.1.1) and to learn from noisy labels. To study the improvement over a noisy baseline, we first investigate a simpler setup, in which the shifter's labels are independent of the algorithm's predictions (section 3.1.2). We then explore the case in which the shifter randomly decides to follow the decision of

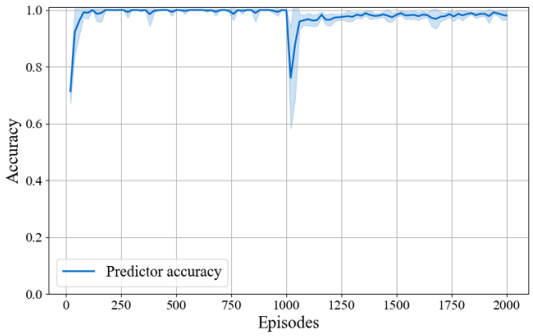


Figure 1. Predictor accuracy against the ground truth labels over the tested episodes in experiment 3.1.1. The mean and error bands indicate successful adaptation to the change in the nominal status at episode 100.

the algorithm, with the probability of this decision being dependent on some estimate of the predictor's accuracy (section 3.1.3). In section 3.1.4, we study the improvements brought to the training by a data-augmentation technique. Finally, we do a test of the algorithm using real data from the LHCb experiment in section 3.1.5.

Finally, we compute the accuracy, F1-score, cumulative reward of the RL policy and the Brier score (as explained in section 2.2.7) across all the experiments and discuss the results. Due to the same expected behavior of most of the metrics used, the following sections we will focus only on the discussion of the accuracy of the agents. Appendix A.2 discusses in detail the rest of the metrics.

3.1.1. Capacity of the algorithm to adapt to changing conditions

We start with a situation where the shifter checks every label. We generated a sequence of 2000 states where the type of nominal distribution is changed from the 1000th state onwards (the specific distributions and parameters used are summarized in table 1 of appendix A.1.1). While this setup tests adaptation to a non-stationary data distribution, it is important to note that it remains strictly within the offline regime. Each data point is treated as an independent event, lacking the temporal state persistence and 'repair' mechanic that defines our online framework (see section 2.1.2). The evolution of the accuracy over the sequence of states is shown in figure 1. As expected, the accuracy first increases as the algorithm learns to distinguish the initial set of nominal and anomalous distributions. Then it drops at state number 1000 due to the change in conditions, and finally recovers after adapting to the new situation. See appendix A.2 for the comparison of the F1-score, cumulative reward of the RL policy and the Brier score in this experiment.

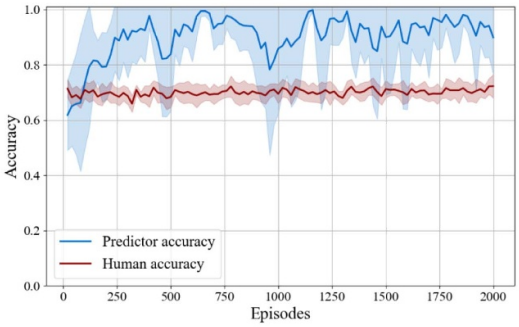
3.1.2. Ability to learn from noisy labels

In this experiment, we generate noisy shifter labels by randomly switching the ground truth of mode of operation with a probability of 30%, to emulate a noisy baseline accuracy of 70%. We generate a sequence of 2000 states with time-independent nominal and anomalous distributions (details in table 1 of appendix A.1.1) and evaluate the evolution of the algorithm's training using these noisy labels. The resulting accuracy of the algorithm's labels, judged against the ground truth of the modes of operation, is shown in figure 2(a). It is compared to the baseline accuracy of the training labels. The algorithm's accuracy surpasses the baseline, demonstrating the ability of the algorithm to learn signal from noisy labels without learning the noise, a classically observed capability of machine learning algorithms (Natarajan *et al* 2013). See appendix A.2 for the comparison of the F1-score.

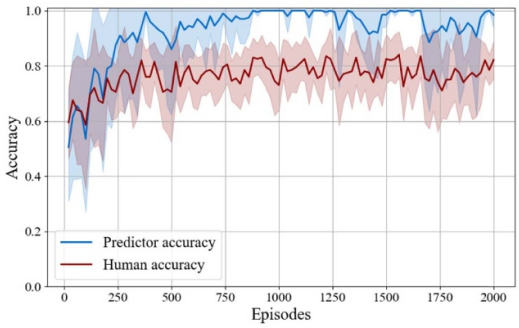
3.1.3. Human-machine mutual feedback

The previous experiment demonstrates the ability of our algorithm to improve over a noisy baseline, as is expected based on the machine learning literature (Natarajan *et al* 2013). We now investigate what happens if the training data, i.e. the labels of the shifters, are influenced by previous outputs of the algorithm. In the real world, this will happen because shifters will have access to the output of the algorithm when making their decision.

For a simplified demonstration, we generate a sequence of 2000 states under the same conditions as in section 3.1.2, except that we modify the decision-making of the shifter as follows. First, the labels of the shifter are a noisy version of the ground truth, with 30% of labels randomly changed, as in section 3.1.2. However, the shifter now gets access to the label provided by the algorithm, as well as a



(a) Shifter decisions before seeing algorithm outputs.



(b) Shifter decisions after seeing algorithm outputs.

Figure 2. Predictor and human accuracy against the ground truth labels over the tested episodes in experiments sections 3.1.2 and 3.1.3. The mean and error bands indicate both the successful ability of the model to learn from human noisy labels and the increment of the human performance when a mutual feedback is given to each other.

proxy for the probability with which the algorithm thinks it is correct, $p^{\text{proxy}}_{\text{correct}}$, here taken to be the softmax of the predictor's output logit with the highest value (see section 2.2.4 for a similar discussion). The shifter now decides to follow the current output of the algorithm, with a probability that depends on $p^{\text{proxy}}_{\text{correct}}$ (details in appendix A.1.2). The predictor is trained with these changed shifter labels. In a further line of research, we may aim to make the model more realistic by having the shifters confidence in their decision also influence the final decision, possibly by having the shifter perform Bayesian inference.

If the shifter's decision was not influenced, this experiment would be identical to the one in section 3.1.2. On the contrary, if the shifter always followed the decision of the algorithm, there would be no further training of the algorithm.

The results are presented in figure 2(b). They demonstrate that in the current setup, the algorithm still trains successfully and is able to improve over the noisy baseline, despite a part of its training data having been generated by a previous version of the algorithm. See appendix A.2 for the comparison of the F1-score.

3.1.4. Data augmentation

In real situations, the training data is scarce. This causes two main problems. First, if the dataset is very imbalanced and only a small number of anomalous examples is available, the algorithm may overfit on them and may fail to generalize to other types of anomalies. Second, the number of iterations required by the algorithm to adapt to new conditions combined with the fact that the data points are collected at fixed time intervals translates into an effective time window that may be too large in cases where quick adaptation is required.

To tackle both challenges, we propose a data-augmentation approach wherein we insert a fixed number of artificially generated data points right after each real data point. Our algorithm is described in appendix A.1.3: it generates histograms based on an automatically built reference for the nominal state and on a large set of predefined types of anomalous deviations. The hyperparameters that control the data augmentation process and their chosen values for the current setup are gathered in table 3.

To test the procedure, we generate a sequence of 150 episodes, changing the type of anomalous distribution after the first 50 episodes and then the type of nominal distribution after the first 100 episodes. The labels provided by the shifter match the ground truth. The specific distributions and parameters

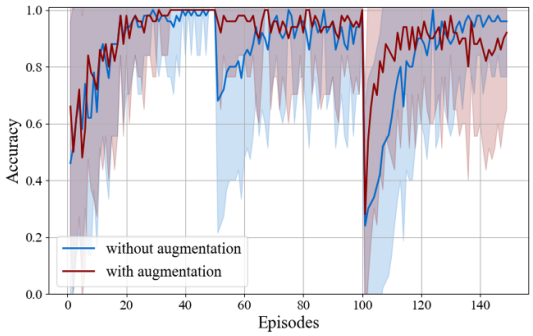


Figure 3. Predictor accuracy against the ground truth labels over the tested episodes in experiment section 3.1.4. The mean and error bands indicate an increase in the model accuracy when data augmentation techniques are applied. This enables the framework to be deployed even with limited data.

used are gathered in table 1. We compare the training in two different cases, with and without data augmentation. The experiments are run 50 times instead of 10 to allow for a better comparison of the shape of the two learning curves. During the data augmentation experiments, we initially observed a policy collapse when abruptly changing nominal conditions, which we attribute to overspecialization on the augmented data. To address this instability, we empirically tested several hyperparameters and found that increasing the entropy coefficient (see section 2.2.5) from 0.5 to 1 was the most effective method to encourage the necessary exploration for stable training.

The results are shown in figure 3. The benefits of data augmentation are evident in two key aspects of the training process. First, at episode 50 where the type of anomaly changes, the augmented model's accuracy remains high, while the non-augmented model experiences a significant drop. This demonstrates that our augmentation strategy helps the algorithm generalize to unseen types of anomalies. Second, after the more challenging change in nominal conditions at episode 100, the augmented training proves to be significantly more stable. While the mean accuracy of both models recovers, the non-augmented training exhibits a much larger variance, with some runs failing to recover at all. The visibly smaller error band for the augmented model indicates a more robust and reliable adaptation process that is less sensitive to the initial random conditions of the training.

It should be noted however that the addition of artificial anomalies obliges the algorithm to solve a more complicated problem than the original one. This would tend to increase the learning time, going against the original goal, but the situation can be solved by an adequate hyper-parameter tuning, as in the case shown here.

3.1.5. Application to offline DQM at the LHCb experiment

The study in this section aims to evaluate how effectively the toy synthetic data captures real-system behavior by comparing algorithmic performance across datasets. The LHCb dataset comprises 2300 data points, featuring a 55 $\times$ larger input dimensionality than the synthetic datasets previously studied and significantly more diverse nominal and anomalous distributions. Technical details regarding the dataset are provided in appendix A.3.1. Ground truth labels are unavailable; instead, labels provided by human shifters are used for training and evaluation. Based on these labels, the dataset is imbalanced, with 1968 nominal and 332 anomalous data points.

The algorithm is trained exclusively on augmented data, generating 100 synthetic data points per real data point sequentially, as detailed in the previous section. Further specifics are outlined in appendix A.3.2. Figure 4 illustrates the evolution of accuracy over time, evaluated on real data in test mode. Given the limited number of data points and the constraints of conducting a single experiment, we applied the same smoothing function as provided by TensorBoard (Abadi *et al* 2016), using a coefficient of 0.95, to depict the accuracy trends. Vertical lines in the figure indicate instances where nominal conditions are known to have changed. Despite the inherent challenges of working with a real-world dataset, the algorithm demonstrates its adaptability to changing conditions, maintaining robust accuracy throughout.

When split by nominal and anomalous data points, as labeled by the shifters, the algorithm achieves an accuracy of 91% and 75%, respectively. The relatively high accuracy for identifying real anomalies is noteworthy, particularly considering that none of these anomalies were included in the training set and that the anomaly generation method employed during data augmentation is overly simplistic. Finally, as the level of human noise in the provided labels is unknown, the reported accuracy values should not be

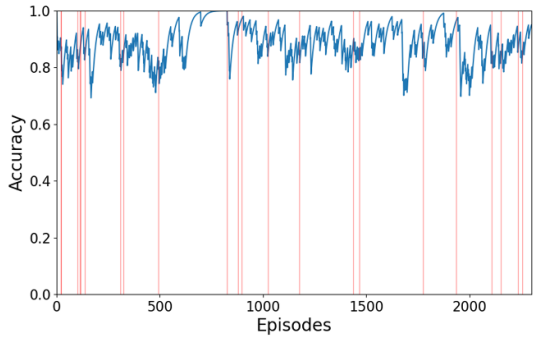


Figure 4. Training accuracy as a function of data collection time for the LHCb dataset. Vertical lines mark data points with changes in nominal conditions. The list of changing conditions indicated in the plot represents a basic assessment by experts of major variations. There may be additional, smaller changes that were not explored in this study.

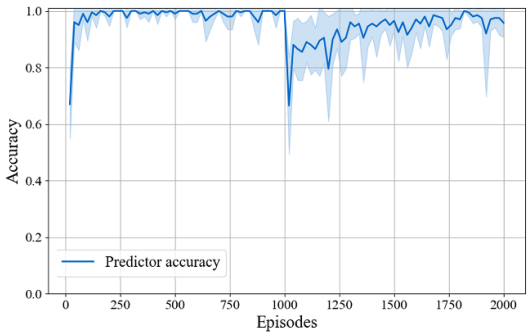


Figure 5. Predictor accuracy against the ground truth labels over the tested episodes in experiment section 3.2. The mean and error bands indicate successful adaptation to the change in the nominal status at episode 100.

overinterpreted. Instead, the emphasis should remain on the key takeaway from this study: even when developed on synthetic data incorporating deliberate simplifications, the core approach is robust enough to generalize to the full complexity of real-world detector data.

3.2. Experiments in the online regime

We generate 2000 episodes, changing the conditions after the first 1000. The distributions and parameters used (same as in section 3.1.1) can be found in table 1 of appendix A.1.1. The shifter's labels, when provided, match the ground truth. The estimated accuracy for the predictor is shown in figure 5, while the fraction of times the checker (estimated in a similar manner as the accuracy) calls the agent is shown in figure 6.

The predictor's accuracy follows the same type of behavior as shown in figure 1, which demonstrates that it trains well and adapts successfully to changing conditions. We observe that the checker calls the shifter mostly at two points in time: at the beginning, when the predictor is not yet trained and its accuracy is consequently poor, and right after the change in conditions, when the predictor's accuracy drops until it learns again. If the accuracy of the predictor is high, the shifter is rarely called. See appendix A.2 for the comparison of the F1-score, cumulative reward of the RL policy and the Brier score in this experiment.

4. Discussion

4.1. General discussion

The experiment presented in section 3.1.3 demonstrates how our RLHF-based strategy enables continuous training of both the algorithm and the shifters based on mutual feedback. This mutual feedback loop increases classification accuracy beyond a shifter-only baseline. In this setup, the shifter judges how much to trust the algorithm's prediction by examining the Predictor's logits for the current state. An alternative approach with similar potential would be to keep a running estimate of the average accuracy achieved by the Predictor in recent states, then use that estimate as the $p_{\text{corr}}^{\text{proxy}}$ variable in the shifter's decision-making process.

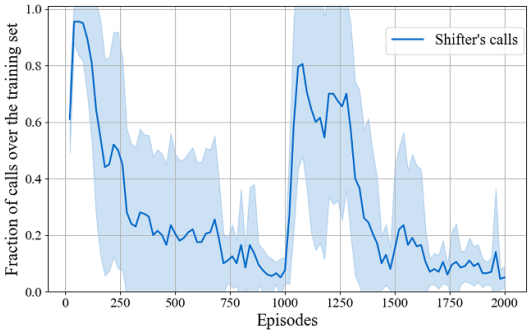


Figure 6. Checker actions to call the shifter over the training episodes in experiment section 3.2. The mean and error bands show that the checker correctly identifies when the predictor requires training, both at the start and following the change in nominal status at episode 100.

Section 3.1.1 demonstrates that the algorithm can adapt to changes in the machine's behavior, suggesting that in practice, with careful hyperparameter tuning, our agent can re-learn at least as effectively as training from scratch. Section 3.1.4 further shows how this adaptation speed can be enhanced via data-augmentation strategies, offering robustness against data scarcity and low sample efficiency in RL (see Kain *et al* (2020) for a related discussion). While our augmentation method is deliberately simple, these techniques could be refined in future work to handle a broader range of anomalies or incorporate more sophisticated generative approaches. For instance, learned models such as generative adversarial networks (GANs) or diffusion models could be trained to produce more realistic nominal and anomalous data samples.

Our synthetic data employs simplified distributions (see appendix A.1.1) to provide a controlled environment for testing the RL agent's core capabilities, such as adaptation and learning from noise. This simplification does not, however, limit the generality of the framework itself. As shown in our study on real LHCb data presented in section 3.1.5, the method generalizes effectively to arbitrary and complex distributions with appropriate hyperparameter tuning. The agent's performance is fundamentally determined by the statistical separability between nominal and anomalous states, rather than the specific form of the underlying data distributions.

Our framework can be deployed in several operational modes, reflecting a progressive path from human-centric oversight to full automation. The first is an *AI-assisted oversight* model, where a human is always present and uses the agent's output as an intelligent aid to improve the accuracy of their manual checks—a scenario explored in our human-machine feedback study (section 3.1.3). A more advanced deployment, and the focus of our online framework, is a *semi-autonomous assistance* model. Here, the agent learns to request human input only when its confidence is low, thereby focusing expert attention on the most ambiguous cases and reducing human fatigue. Finally, in a potential future application, a *full-automation* mode could be achieved if the system proves sufficiently robust. In this scenario, the agent would substitute the shifter entirely and directly alert relevant detector experts when intervention is needed. This represents the most significant long-term impact, where shifter-hours could be fully repurposed for other tasks. Such an optimization is learned directly from the reward signal; while we focus on a simple reward scheme, other approaches could assign rewards proportional to the collected amount of nominal data, penalized by the resource or operational costs of the relevant actions.

Our offline studies with noisy labels and data augmentation can, in principle, be extended to the online regime. Although data augmentation in the online setting faces the additional complexity of time dependencies between consecutive states, we anticipate it could still improve sample efficiency. Investigating these extensions is left for future work.

Regarding computational requirements (section 2.1.3), we find that training remains relatively fast. Most training iterations take well under a second for the non-augmented case, and only a few seconds with data augmentation. Since histograms are typically generated every few minutes in DQM applications, training time should not be a bottleneck for real-world deployments.

Finally, the study with real-world LHCb data in section 3.1.5 demonstrates generalization from synthetic data to real detector conditions, contingent on suitable hyperparameter tuning and careful data processing. This proof-of-concept can be expanded by identifying more precise ground truth labels in consultation with detector experts, exploring richer forms of anomaly generation, and further optimizing hyperparameters.

Although tested here on LHCb, the approach itself remains broadly applicable to a variety of DQM tasks both inside and outside particle physics. Our framework of combining RL with human feedback, augmented data, and dynamic anomaly detection can be readily adapted to other large-scale, continuously evolving systems.

4.2. Limitations

The main remaining challenge that we foresee when bringing the presented approach to real application concerns the robustness of the training against shifters' biases. These biases may arise from a mismatch between the average knowledge of the shifters and that of the small set of detector experts in the experiment. One way to address this is to build 'reference' histograms that reflect what a nominal' histogram would look like based on the shifters' decision, and then have these references checked with a certain frequency by detector experts so that biases in the shifters' decisions can be addressed. The construction of such a reference is already part of our data augmentation approach.

A further limitation of this work is the lack of an exhaustive hyperparameter optimization. The primary goal of this paper is to demonstrate the viability of the proposed RLHF approach, not to achieve optimal performance on a specific benchmark. For this reason, we used a single, reasonable baseline configuration for most synthetic studies and performed only a superficial, empirical tuning for the more complex real-data application. While we found the approach to be robust, we acknowledge that a more rigorous, application-specific optimization and a detailed study of the algorithm's stability with respect to its hyperparameters are important steps for future work and real-world deployment.

4.3. Conclusions

We have presented a proof-of-concept demonstration of RLHF to automate DQM in large-scale particle physics experiments. By reframing DQM as an RL problem, we move beyond static anomaly detection to a dynamic, adaptive policy that learns online or offline, robustly handles label noise, and judiciously requests human oversight. Our experiments on synthetic data reveal strong adaptability to shifts in nominal and anomalous distributions, the capacity to learn from noisy or self-referential labels, and the benefits of augmented training data. Initial tests on a real LHCb dataset suggest that this approach can generalize to practical conditions.

We envision future work that uses more advanced data-augmentation techniques, extends to additional control actions such as automatically triggering repairs or reconfigurations and incorporates cost functions that reflect real operational overhead. Ultimately, our approach could help large experiments gather higher-quality data more reliably and with fewer resources, opening the door to broader automation and optimization in scientific control rooms.

Data availability statement

In accordance with the LHCb Collaboration's External Data Access Policy, the data cannot be made publicly available upon publication because they are owned by a third party and the terms of use prevent public distribution. More information can be found in Clarke Peter *et al* (2013).

Acknowledgment

We express our gratitude to the LHCb collaboration for providing the data used to check the performance of the algorithm. The work of OJP has been supported by CERN openlab (openlab.cern) jointly with the University of Zurich. JGP is supported by the U.S. National Science Foundation under Grant No. 2411204. TL has been partially supported by the SNSF Advanced Grant *TMAG-2_209263*. Finally, SK has been supported as part of the SK's project *Uncovering the lepton generation gap* (with project number VI.Veni.202.004) of the Veni research program, which is partly financed by the Dutch Research Council (NWO).

Ethics statement

This paper presents a proof-of-concept for a new approach to DQM in a fundamental research setting, specifically large-scale particle physics experiments. As such, the scope of any potential positive or negative outcomes primarily remains within the realm of scientific data-taking and does not directly extend to broader societal or ethical consequences. The method deals only with aggregated detector data-no personal or sensitive information-and therefore poses minimal concerns for privacy or fairness. No research was conducted on human subjects, human data or tissues, or animals.

Author contributions

Olivia Jullian Parra 0009-0009-8373-1282
Conceptualization (supporting), Formal analysis (supporting), Investigation (equal), Methodology (equal), Software (lead), Validation (equal), Visualization (lead), Writing – original draft (equal), Writing – review & editing (equal)

Julián García Pardiñas
Conceptualization (lead), Data curation (lead), Formal analysis (lead), Investigation (equal), Methodology (equal), Project administration (lead), Resources (lead), Software (supporting), Supervision (lead), Validation (lead), Visualization (equal), Writing – original draft (lead), Writing – review & editing (lead)

Lorenzo Del Pianta Pérez
Conceptualization (equal), Investigation (equal), Methodology (equal), Software (supporting)

Maximilian Janisch
Conceptualization (supporting), Investigation (supporting), Writing – original draft (equal), Writing – review & editing (equal)

Suzanne Klaver
Methodology (supporting), Resources (equal), Writing – original draft (supporting), Writing – review & editing (supporting)

Thomas Lehéricy 0000-0001-7508-7233
Conceptualization (supporting), Investigation (supporting), Writing – original draft (equal), Writing – review & editing (equal)

Nicola Serra 0000-0002-5033-0580
Conceptualization (equal), Investigation (supporting), Supervision (supporting)

Appendix

A.1. Technical details of the synthetic DQM study

A.1.1. Synthetic dataset

The synthetic data points used in the different experiments in this paper take the form of one-dimensional histograms of 100 bins each. The length of the datasets is 2000 histograms in sections 3.1.1–3.1.3 and 3.2, and 150 for section 3.1.4. The histograms are based on a certain underlying distribution, f, that we choose to be either Gaussian, f_g, or exponential, f_e, parametrized as follows:

$$f_g(x) = \frac{c_{g1}}{\sqrt{2\pi\, c_{g3}^2}} \cdot \exp\left(-\frac{(x - c_{g2})^2}{2c_{g3}^2}\right), \tag{1}$$

$$f_e(x) = c_{e1} \cdot c_{e2} \cdot \exp(-c_{e2}x), \tag{2}$$

with c_{g1}, c_{g2}, c_{g3}, c_{e1} and c_{e2} being constant parameters. We fix the total mass $c_{g1} = 10$ and $c_{e1} = 1$ for both anomalous and nominal histograms. Fixing these normalization constants ensures that the total mass of the histograms does not change when other shape parameters (e.g. c_{g3} or c_{e2}) are varied. This forces the algorithm to learn from the distributional shape rather than the overall number of events, which is the intended focus of this study. Figures 7 and 8 show examples of the simulated types of histograms a shifter could find in our experiment setup following f_g or f_e distributions.

The generation of each histogram is done bin by bin, using the function's value at the bin's center and adding a random Gaussian fluctuation to emulate stochastic effects. For the experiments in this paper, the Gaussian noise uses a constant width of 0.01 for all bins. We acknowledge this is a simplification, as the uncertainty in a simple unnormalized histogram is typically Poisson-based, leading to higher relative variance in low-yield bins. However, in many real-world DQM applications, the sources of uncertainty are more complex. For instance, histograms are often normalized to a unit area, as is done for the LHCb data in this study. If the total number of events varies between runs, this normalization leads to non-trivial, non-Poissonian error propagation. Furthermore, some monitored quantities like differential efficiencies have inherently complex uncertainties, and a single 'nominal' ground truth label may encompass multiple acceptable operational sub-states (e.g. inoffensive noisy pixels that do not

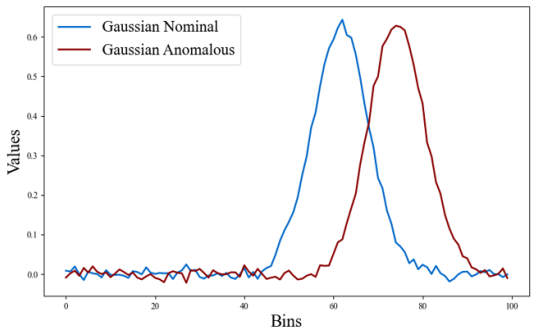


Figure 7. Example of Gaussian histograms illustrating the different distributions observed: one corresponding to the nominal status and one to the anomalous status.

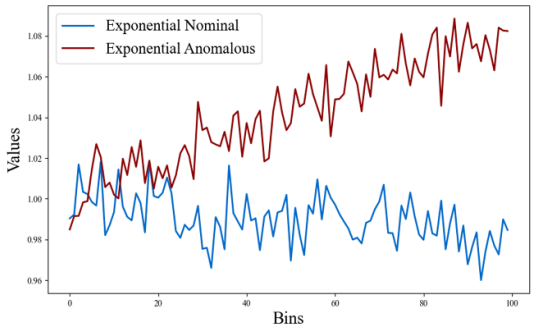


Figure 8. Example of exponential histograms illustrating the different distributions observed: one corresponding to the nominal status and one to the anomalous status.

Table 1. Distributions used for the synthetic DQM experiments. All the histograms are composed of 100 bins.

Experiment	Episode range	Nominal function	Nominal params	Anomalous function	Anomalous params
Sections 3.1.1 and 3.2	$[0, 999]$ $[1000, 1999]$	f_e f_g	$c_{2e} = -0.0002$ $c_{g2} = 10, c_{g3} = 30$	f_g f_g	$c_{g2} \in [10, 80], c_{g3} = 30$ $c_{g2} \in [10, 80], c_{g3} = 30$
Sections 3.1.2 and 3.1.3	$[0, 1999]$	f_g	$c_{g2} = 10, c_{g3} = 30$	f_e	$c_{e2} \in [-0.005, 0.005]$
Section 3.1.4	$[0, 49]$ $[50, 99]$ $[100, 149]$	f_g f_g f_e	$c_{g2} = 10, c_{g3} = 30$ $c_{g2} = 10, c_{g3} = 30$ $c_{e2} = 0.0002$	f_e f_g f_g	$c_{e2} \in [-0.005, 0.005]$ $c_{g2} \in [0, 100], c_{g3} = 40$ $c_{g2} \in [0, 100], c_{g3} = 40$

impact physics performance). Given these complexities, we adopt a generic, fixed-width noise model as a reasonable proxy for the combined stochastic effects present in a real system. The goal of our synthetic study is not to perfectly replicate a specific noise source, but to test the algorithm's ability to learn in a noisy environment, a robustness which is ultimately demonstrated on real data.

Either the Gaussian or exponential distributions can represent a nominal or an anomalous state within a given experiment. If they are representing a nominal state, the distribution shape parameters will remain fixed. If they are representing an anomalous state, some of their values will be sampled from a certain range. The values chosen for all the experiments described in this paper can be found in table 1.

A.1.2. Trust function

In this appendix, we describe the function f which assigns to $p^{\text{proxy}}_{\text{correct}}$, as introduced in section 3.1.3, the probability with which a shifter having access to the output of the algorithm will randomly decide to replace their decision with that of the algorithm.

In general, it seems reasonable to assume that this function is monotonically increasing. The shifters may not use the decisions of the algorithm at all if $p^{\text{proxy}}_{\text{correct}}$ is very low. Furthermore, in order to allow for

Table 2. Parameters used in the trust function.

Name	Chosen value
s	0.95
p_0	0.7
w	0.02

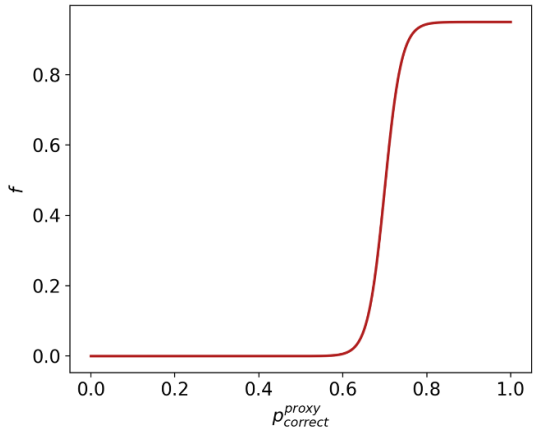


Figure 9. Function used to model the shifter's probability of trusting the algorithm.

shifters to not strictly follow the algorithm even if the latter is very confident in its classification (that is, when $p^{\text{proxy}}_{\text{correct}} \approx 1$), we saturate the probability of having a shifter replace its decision by that of the algorithm at some $s < 1$. For our demonstration, we choose the function f as follows:

$$f\left(p^{\text{proxy}}_{\text{correct}}\right) = \frac{s}{1 + \exp\left(-\left(p^{\text{proxy}}_{\text{correct}} - p_0\right)/w\right)}, \tag{3}$$

where s is the value to which the probability saturates, p_0 is the point at which the majority of shifters adapt the decision of the algorithm, and w is a parameter that controls the width of the region in which the probability increases in a quasi-linear way. The heuristically chosen values for those parameters are written up in table 2, and the resulting function is shown in figure 9.

A.1.3. Data augmentation algorithm

This section describes the process to generate an artificial histogram, represented by the input vector z_{bin} and the corresponding artificial label, nominal or anomalous. The procedure includes the construction and continuous update of a reference histogram, represented by a vector of bin centers, μ_{bin}, and a vector of bin uncertainties, σ_{bin}. This histogram takes information from the input vector, x_{bin}, of the original histograms in the training dataset that are labeled as nominal. The update of the reference histogram over time is based on the exponentially weighted moving average (EWMA) approach (Roberts 1959), to allow the adaptation to changing conditions over time.

The structure of the algorithm is described in algorithm 1 and its hyperparameters are described in table 3. Two different parameters, α_μ and α_σ, are used to control separately the time evolution of the bin centers and uncertainties of the reference histogram. Adjusting their values allows to change the relative impact on the reference histogram of more recent nominal histograms in the training dataset compared to older ones. The data augmentation process is started only after a certain number of training steps, to allow time for the reference template to sufficiently stabilize. All the parameters have been moderately tuned by hand for the experiment described in section 3.1.4. An in-detail hyperparameter optimization is out of the scope of this paper.

A.1.4. Experiment setup

The simulated environment supports two distinct interaction regimes: offline and online, each reflecting different real-world anomaly detection settings. These regimes are implemented within a Toy_environment class, which inherits from the Gym-compatible base class. The selection of the regime is controlled via a configuration parameter.

Table 3. Parameters used in the data augmentation algorithm.

Name	Description	Chosen value
α_μ	EWMA parameter for bin centers	0.99
α_σ	EWMA parameter for bin uncertainties	0.5
AugmStartPoint	Number of the first training step with augmentation	30
AugmFactor	Number of augmented states for each original one	499
AugmAnomProb	Probability that an augmented state is an anomaly	50%
MinNumVars	Minimum number of variations	1
MaxNumVars	Maximum number of variations	50
MinVarSize	Minimum number of bins in each variation	1
MaxVarSize	Maximum number of bins in each variation	100
MinNumStdDev	Minimum number of standard deviations	5
MaxNumStdDev	Maximum number of standard deviations	20

Algorithm 1. Training with data augmentation.

Initialize μ_{bin} to 0
Initialize σ_{bin} to 1
for each histogram in training dataset **do**
  Do one training step using x_{bin} and its label
  if the current histogram is nominal {Update the reference histogram} **then**
    $\mu_{bin} \leftarrow \alpha_\mu \cdot x_{bin} + (1 - \alpha_\mu) \cdot \mu_{bin}$
    $\sigma_{bin} \leftarrow \sqrt{\alpha_\sigma \cdot (x_{bin} - \mu_{bin})^2 + (1 - \alpha_\sigma) \cdot \sigma_{bin}^2}$
  end if
  if the training step is $\geqslant$ *AugmStartPoint* {Start the data-augmentation process} **then**
    for *AugmFactor* iterations **do**
      Generate an augmented histogram $z_{bin} \sim N(\mu_{bin}, \sigma_{bin})$
      Sample whether the state will be an anomaly, with probability *AugmAnomProb*
      Set the histogram label according to whether it is anomalous or not
      if the state is an anomaly **then**
        for a number of iterations in [*MinNumVars*, *MaxNumVars*] **do**
          Sample an integer variation size in [*MinVarSize*, *MaxVarSize*]
          Sample the position of the first (left-most) bin in the variation
          Sample a real scaling factor, *SF*, in [*MinNumStdDev*, *MaxNumStdDev*]
          Randomly make the *SF* negative with a 50% probability
          Modify every bin content in the variation, bin', as $z_{bin'} \leftarrow z_{bin'} + SF \cdot \sigma_{bin'}$
        end for
      end if
      Do one training step using z_{bin} and its label
    end for
  end if
end for

Model configuration. We use standard feedforward neural networks to represent policies and value functions. In the offline regime we will only have one agent: the predictor. The predictor is in charge of deciding whether the mode of operation in a given state is nominal or anomalous. On the other hand, in the online regime another agent is required: the checker. The checker has to examine the predictor's response for a given data point and decide whether to obtain a label from an external shifter or not.

Each agent follows an actor-critic architecture, where the actor selects actions based on a learned policy and the critic estimates the expected return. In the case of the predictor's output, a two-dimensional vector represents the logits for the nominal and anomalous classes, used for both classification and estimating prediction confidence. On the other hand, the checker outputs a binary decision indicating whether to query the shifter (i.e. request a label) or not, based on the predictor's uncertainty and its own learned policy. The final decisions for both agents are taken based on the highest softmax output.

All the offline environments use the same model architecture, which is chosen to balance capacity and stability in offline settings. Table 4 below summarizes the architecture:

As for the online environment, table 5 below summarizes the architecture:

Table 4. Model architecture used for all offline experiments.

Component	Model architecture
Policy network	Two-layer MLP (10 units per layer, ReLU activation)
Value network (Critic)	Two-layer MLP (10 units per layer, ReLU activation)

Table 5. Model architecture used for online experiment.

Component	Model architecture
Policy network	Two-layer MLP (32 and 16 units per layer, ReLU activation)
Value network (Critic)	Two-layer MLP (10 units per layer, ReLU activation)

This architecture is widely used in RL due to its simplicity, stable convergence, and sufficient expressivity for low- and mid-dimensional control tasks. It performs well across a variety of RL benchmarks and is appropriate for the scale of the environments used in our experiments. More complex architectures have been tried, resulting on similar results with less computational efficiency.

Simulations setup for the offline regime. In this regime, the environment mimics a DQM offline setting where each histogram is independently drawn. Thus, at each step:

- A histogram is sampled either from a ground-truth distribution or an anomaly distribution based on a fixed probability p_{anom} (in our case 0.3, explained in section 2.1).
- The type of histogram (anomalous or normal) is sampled i.i.d. from the configured distributions.
- The predictor agent receives the histogram, performs a classification action, and obtains a reward.
- Label noise is introduced via the $p_{\text{human error}}$ parameter to simulate imperfect human supervision.

The dataset in the offline setup is thus composed of histograms that are ordered chronologically. In what follows, we simplify the model by assuming that the state of each data point (whether it is nominal or anomalous) is an independent event. This means there are no time correlations, such as a single fault state persisting across multiple consecutive data points. Thus, the notion of 'time' in our formulation does not exactly refer to real time or to shifter input, but rather to the sequence of steps in the Markovian process. Additionally, while each histogram sample is drawn independently, the underlying distribution that defines what is considered anomalous versus nominal can evolve over the sequence of steps. This makes the classification condition a changing one, allowing one to test the agents on how well they adapt to it (see sections 3.1.1, 3.1.4 and 3.2).

Note that for our synthetic data studies, each data point is generated as anomalous with a fixed probability, p_{anom}, which we set to 0.3. This value was chosen to create a reasonably balanced dataset for our proof-of-concept experiments. In real-world applications, this anomaly fraction is an unknown characteristic of the environment. For context, the fraction estimated from the (potentially imperfect) human-provided labels in the LHCb dataset is approximately 14% (see section 3.1.5), which suggests that our choice of 0.3 is suitable for these studies. A significantly lower fraction in a real application would influence model tuning and could heighten the need for strategies like data augmentation (see section 3.1.4).

As for the setup in experiments sections 3.1.2 and 3.1.3 (related to noisy labels and human-machine mutual feedback), we add a noisy label using $p_{\text{human error}}$. We set this probability to 0.3 as well as a prudent upper-bound estimate. This value is conservative enough to realistically capture the possibility of occasional calibration or labeling mistakes in complex data handling, without being so large as to overwhelm instrument-related uncertainties. In this way, the monitoring system remains cautious about human contributions to data quality while still allowing the underlying physics processes to dominate the analysis.

Simulations setup for the online regime. In contrast, the online regime simulates a continuous data stream where histograms are temporally dependent. In this setting:

- At the beginning, the histogram is sampled either from a ground-truth distribution or an anomaly distribution based on a fixed probability p_{anom}.
- The agent predictor receives the histogram and performs a classification action. Then, the checker has to examine the predictor's response and decide whether to obtain a label from an external shifter or not. If called by the checker, a shifter will provide a label that can be used to train the predictor. If

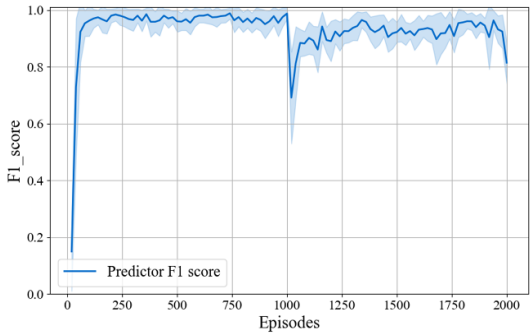


Figure 10. Predictor F1 score against the ground truth labels over the tested episodes in experiment section 3.1.1. The mean and error bands indicate successful adaptation to the change in the nominal status at episode 100.

not called by the checker, in the online regime, there is a fixed probability p (that we set to 0.15) with which a shifter will provide a label anyway.

- Unlike the offline regime, if an anomaly is introduced and not detected by the agent, it will persist across subsequent steps until it is correctly classified. This models scenarios where faults remain active until addressed.
- Once an anomaly is correctly detected, the environment reverts to a probabilistic sampling strategy: with a fixed probability p_{anom}, a new anomaly may be introduced at the next step. If no anomaly is sampled, the system resumes generating normal histograms.
- This feedback-dependent persistence introduces temporal correlations and forces the agent to adapt dynamically, rather than relying solely on i.i.d. statistics.

The key difference between the two regimes is the temporal dependency between the type of histogram drawn at the next step. In one hand we set up an offline regime, where each histogram is independent and identically distributed (i.i.d.), suitable for supervised classification. On the other hand, the online regime, where the environment simulates a non-stationary process with temporal correlations, more representative of real-time monitoring systems. This dual-mode simulation allows for flexible benchmarking of anomaly detection algorithms under both batch and streaming conditions.

A.2. Results comparison of the synthetic DQM study

A.2.1. Capacity of the algorithm to adapt to changing conditions

As described in section 3.1.1, it has been proven through the evolution of the accuracy over the sequence of states that, as expected, performance metrics will first increase as the algorithm learns to distinguish the initial set of nominal and anomalous distributions. Then, we expect a drop at state number 1000 due to the change in conditions, and finally a recovery after adapting to the new situation. Figures 10 and 11 show as well this adaptation of the agent to the new changing condition, as both experience a drop in the state number 1000 followed by a recovery after retraining. On the other hand, the Brier score (figure 12) decreases at first (demonstrating high confidence in the predicted actions once the agent is correctly trained), and it experiences a drop of its confidence at state number 1000, demonstrating as well the need of retraining after a new unknown condition is found. Finally, when retrained again, the confidence increases again together with its accuracy, decreasing the score towards 0.

A.2.2. Ability to learn from noisy labels

As demonstrated in section 3.1.2, together with the resulting accuracy of algorithm, the F1-score also surpasses the baseline, demonstrating the ability of the algorithm to learn signal from noisy labels without learning the noise. Figure 13 shows this ability.

A.2.3. Human-machine mutual feedback

On the same line of research, section 3.1.3 presents the ability to improve its accuracy over the noisy baseline even when part of its training data has been generated by a previous version of the algorithm. To ensure the robustness of this findings, F1-score is also computed and compared to the noisy baseline in the same way as the accuracy was (see figure 14). Both results show the ability of the algorithm to learn from noisy labels even in the presence of human-machine mutual feedback.

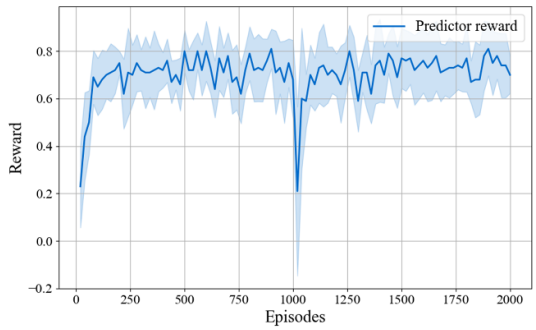


Figure 11. Predictor accumulated training rewards against the ground truth labels over the tested episodes in experiment section 3.1.1. The mean and error bands indicate successful adaptation to the change in the nominal status at episode 100.

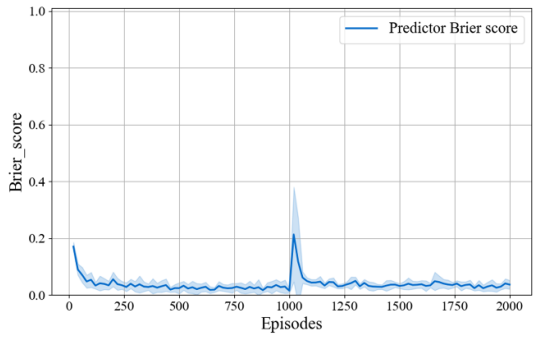


Figure 12. Predictor Brier score against the ground truth labels over the tested episodes in experiment section 3.1.1. The mean and error bands indicate successful adaptation to the change in the nominal status at episode 100.

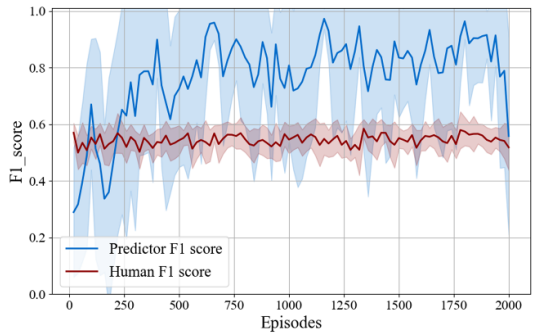


Figure 13. Predictor and human F1 score against the ground truth labels over the tested episodes in experiment section 3.1.2. The mean and error bands indicate the successful ability of the model to learn from human noisy labels.

A.2.4. Experiments in the online regime

To support the conclusions drawn in section 3.2, we computed the F1-score and the training cumulative reward of the RL predcitor's policy to check the ability of the predictor to adapt to changing conditions. As figures 15 and 16 show, the predictor is successfully trained also in this regime.

The F1-score follows the expected behavior seen already in section 3.1.1: it first increases (as the algorithm learns to distinguish the initial set of nominal and anomalous distributions) and then, a drop can be seen at state number 1000 due to the change in conditions, with its posterior recovery after adapting to the new situation. Figure 15 shows this behavior.

But where the true success of both agents is seen when we check both the reward trajectories of the predictor agent and its Brier Score. Figures 16 and 17 reveal the interplay between the predictor and the checker in two distinct phases: an initial training and a posterior retraining after a change in nominal conditions (correctly detected by the checker).

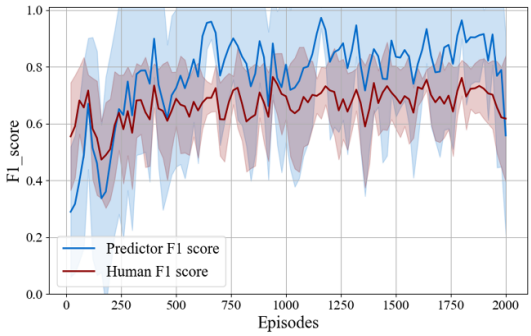


Figure 14. Predictor and human F1 score against the ground truth labels over the tested episodes in experiment section 3.1.3. The mean and error bands indicate the successful increment of the human performance when a mutual feedback is given to each other.

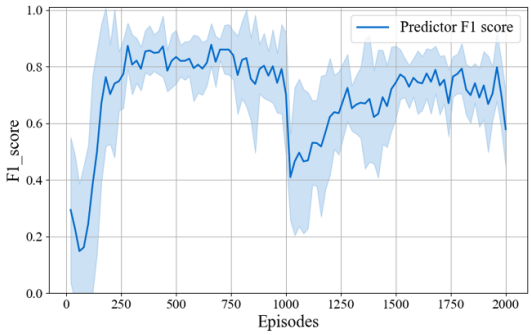


Figure 15. Predictor F1 score against the ground truth labels over the tested episodes in experiment section 3.2. The mean and error bands indicate successful adaptation to the change in the nominal status at episode 100.

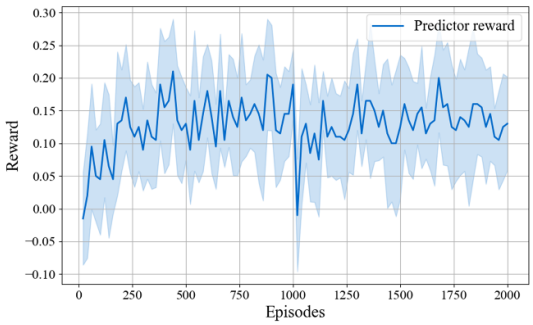


Figure 16. Predictor accumulated training rewards against the ground truth labels over the tested episodes in experiment section 3.2. The mean and error bands indicate successful adaptation to the change in the nominal status at episode 100.

During the first training phase, the predictor learns to make correct predictions under the original nominal conditions thanks to the checker's first calls to the shifter and then, its rewards stabilize, with occasional positive rewards arising from sparse random checks.

At episode 1000, we modified the nominal condition to test whether the checker correctly detects deviations. Following this change, the predictor's rewards temporarily drop, indicating that the checker is actively calling the shifter whenever the predictor fails under the new conditions. This behavior can be seen as well based on figure 17, where the predictor's Brier Score increases (representing a random prediction from the predictor's side), demonstrating its uncertainty, taken then by the shifter and used to call the shifter (see figure 6). The drop in the predictor's reward is linked to both the increase in the Brier score and shifter calls from the checker. This confirms that the checker correctly identifies deviations from the expected behavior and provides corrective feedback. Then, during this retraining phase, the predictor gradually adapts to the new conditions, and the rewards recover. However, as the predictor

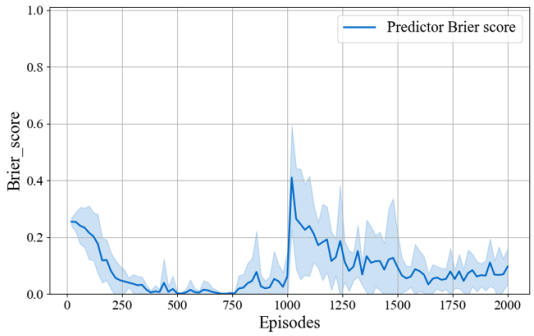


Figure 17. Predictor Brier score against the ground truth labels over the tested episodes in experiment section 3.2. The mean and error bands indicate successful adaptation to the change in the nominal status at episode 100.

improves, the shifter is invoked less frequently (see figure 6), and the recovered rewards primarily originate from sparse random checks rather than shifter calls, as expected.

Note that, the main change with respect to the offline regime resides in the maximum reward acquired by the predictor. Even when the predictor achieves high accuracy or F1-score in either phase, the running average reward remains around 0.15. This reflects the sparse and conditional nature of the reward: steps without random checks or shifter calls yield zero reward, while intermittent correct predictions during random checks contribute positively. These dynamics collectively demonstrate that the checker is correctly performing in both training and retraining: initially to guide learning under standard conditions, and subsequently to detect and respond to changes in the environment, ensuring effective adaptation of the predictor to new nominal conditions.

These results allows us to ensure the efficiency of the algorithm in this type of regime.

A.3. Technical details of the LHCb offline DQM study

A.3.1. Dataset description

The dataset comprises 2300 data points, each containing information relevant to monitor the performance of the various LHCb detector subsystems and the (particle reconstruction) software trigger during data-taking operations at the LHC. Each data point consists of a curated subset of histograms representing critical data features. These monitor a wide range of quantities, from low-level hardware statuses like detector high voltages and temperatures to high-level physics performance metrics such as track reconstruction residuals used for alignment, particle identification efficiencies, and invariant mass distributions of well-known particles like the J/ψ meson (Alves *et al* 2008). The data for each point is accumulated over a specific time interval, ranging from a few minutes to approximately one hour. These data points are sequentially ordered by collection time, providing a chronological record of the monitoring process.

Combined, the dataset represents several months of operational time during particle collisions recorded by the LHCb experiment in 2024. Although this period constitutes only a small fraction of the experiment's lifetime, it captures a phase characterized by significant changes in nominal operation. These changes followed the commissioning of a major machine and software upgrade (Aaij *et al* 2024), making the dataset particularly valuable for studying the algorithm's adaptability.

Each data point consists of 73 one-dimensional histograms, with an average of 75 bins per histogram (ranging from 2 to 200 bins). To ensure consistency, each histogram is independently normalized to have a unit area. The normalized bin contents from all histograms within a data point are then concatenated to form a single vector representation with 5501 bins.

A.3.2. Data augmentation and training configuration

A first-level hyperparameter optimization was conducted using Optuna, with the objective of maximizing the balanced accuracy on the dataset. The accuracy was assessed in testing mode, calculated sequentially after training on each data point. The optimization was performed only superficially and on the same dataset, as the goal of this work is not to benchmark absolute accuracy but to demonstrate the algorithm's applicability to real data while deriving conclusions consistent with synthetic studies. The resulting optimized parameter values are detailed below.

The following parameter values were adopted for data augmentation (see table 3 for parameter definitions): $\alpha_\mu = 0.9$, $\alpha_\sigma = 0.5$, $AugmStartPoint = 20$, $MinNumVars = 4$, $MaxNumVars = 19$, $MinVarSize = 5$, $MaxVarSize = 192$, $MinNumStdDev = 3$, and $MaxNumStdDev = 11$. For each data point, 100 synthetic

samples were generated, with a probability of 50% for being anomalous. Each synthetically generated histogram was normalized to unit area after augmentation. Policy updates were performed in subbatches of 50 samples, with 30 iterations of training applied per subbatch. A learning rate of 1×10^{-4} and an entropy coefficient of 0.7 were used throughout the study.

The actor and critic networks employed in this work both have a fully connected architecture with the following layer configuration: [10 000, 500, 10]. This setup is specifically designed to efficiently handle the high dimensionality of the input data (5501 bins) while maintaining computational feasibility. These settings are summarized and contrasted with the synthetic study configuration in table 6 in appendix A.5.

A.4. Review of RL and PPO

In this section we review the basic theory of PPO, starting with an elementary recall of RL, going over basic algorithms aiming to find optimal policies, so-called policy gradient methods (PGMs), and finally introducing PPO.

A.4.1. Formal setting for RL

Recall that a RL agent is tasked with maximizing a reward in a Markov decision process. A *Markov decision process* is a tuple $(\mathcal{S}, \mathcal{A}, \rho, \mathbb{P}_0)$ consisting of a measurable space $\mathcal{S}$, called the set of *states*, as well as a measurable space $\mathcal{A}$, called the set of *actions*. ρ is a Markovian transition kernel from $\mathcal{S} \times \mathcal{A}$ to $\mathbb{R} \times \mathcal{S}$, called the *environment dynamics*, such that for every $s \in \mathcal{S}$ and $a \in \mathcal{A}$, $\rho(\cdot, \cdot \mid s, a)$ is a probability measure on $\mathbb{R} \times \mathcal{S}$ defining the joint probability distribution of the reward and the next state if one starts in state s and performs action a. $\mathbb{P}_0$ is a probability distribution on $\mathcal{S}$. We start at a state S_0 which is $\mathbb{P}_0$-distributed. $\mathbb{N} = \{0, 1, \dots\}$ is the set of natural numbers.

At every time $t \in \mathbb{N}$, the agent (in state S_t) performs an action that brings it to a state S_{t+1}, sampled according to a *policy* π_t, and gets a reward R_t. The policy π_t is a Markovian transition kernel from $\mathcal{S}$ to $\mathcal{A}$. For every $s \in \mathcal{S}$, $\pi_t(\cdot \mid s)$ defines the probability distribution of which action to take if at time t one is in state s.

For $t \in \mathbb{N}$, we define recursively random variables A_t, R_t, S_{t+1} which satisfy that the conditional distribution of A_t given S_t is $\pi_t(\cdot \mid S_t)$ and that

$$(R_t, S_{t+1}) \mid S_t, A_t \sim \rho(\cdot, \cdot \mid S_t, A_t). \tag{4}$$

Remark A.1 (Formal definition of the A_t, R_t, S_{t+1}). To give a formal definition, we may proceed as follows: we fix first a probability space with two families of random variables $(X_{t,s})_{t \in \mathbb{N}, s \in \mathcal{S}}$ and $(Y_{t,s,a})_{t \in \mathbb{N}, s \in \mathcal{S}, a \in \mathcal{A}}$ which we assume to be jointly independent, and which we assume to satisfy $X_{t,s} \sim \pi_t(\cdot \mid s)$ as well as $Y_{t,s,a} \sim \rho(\cdot, \cdot \mid s, a)$ for all $t \in \mathbb{N}, a \in \mathcal{A}, s \in \mathcal{S}$. (The existence of such a space is guaranteed for instance by the Andersen–Jessen theorem.) We then set, recursively,

$$A_t \overset{\text{Def.}}{=} X_{t,S_t} \qquad (R_t, S_{t+1}) \overset{\text{Def.}}{=} Y_{t,S_t,A_t} \tag{5}$$

for $t \in \mathbb{N}$.

The random variable A_t is the action one takes at time t, while S_t is the state at time t and R_t is the reward obtained at time t.

Note that all of these random variables depend on the choice of the policies π_t. The goal of the agent is now to choose policies which maximize the cumulative expected reward

$$\sum_{t \in \mathbb{N}} \gamma^t \mathbb{E}(R_t), \tag{6}$$

where $\gamma \in \mathbb{R}_{\geqslant 0}$ is a so-called *discounting factor*.

Definition A.2 (State value function). The *state value function at time t* assigns to a state $s \in \mathcal{S}$ and time $t \in \mathbb{N}$ the expected future cumulative reward if we are in this state:

$$V(t, s) \overset{\text{Def.}}{=} \sum_{t' \in \mathbb{N} \cap [t, \infty)} \gamma^{t'} \mathbb{E}(R_{t'} \mid S_t = s). \tag{7}$$

(In a finite horizon setting we may only sum over all t' up to some time $T \in \mathbb{N}$.)

Summarizing, there are two main parts of a RL algorithm:

Policy The policies π_t defining what action to take at time t for each state;

State value function The state value function V giving the expected future cumulative reward at time t for each state.

A.4.2. PGMs

To illustrate how optimization of the policies proceeds, we first describe so-called *PGMs*. PGMs have a parametrized family of policies $(\pi_t^\theta)_{\theta\in\Theta}$, where Θ is some index set. The goal of PGMs is to update the parameters θ in the direction of the gradient of the expected return, thus optimizing the parameters of the policy using a form of gradient ascent. More formally, note that R_t from appendix A.4.1 can be considered a function of θ. PGM algorithms such as REINFORCE, introduced by Williams (1992), iterate by performing policy ascent. Starting at some $\theta_0 \in \Theta$, one then iterates as follows:

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta \mathbb{E}\left(R_t\right), \tag{8}$$

where $\alpha \in \mathbb{R}_{\geqslant 0}$ is the *learning rate*.

Remark A.3 (Summing over time). In the current formulation, we are optimizing the policy of a single time step t only. One can formulate the same idea by parameter-sharing across all policies $(\pi_t)_{t\in\mathbb{N}}$ and taking the derivative of a discounted reward $\sum_{t\in\mathbb{N}} \gamma^t R_t$ or a finite-horizon reward $\sum_{t=0}^{T} R_t$ instead. For ease of exposition, we limit ourselves to one time step but this Remark is valid throughout the rest of the section.

In order to compute $\nabla_\theta \mathbb{E}(R_t)$, REINFORCE employs the following Lemma.

Lemma A.4 (Log-derivative trick). *Let $(p^\theta)_{\theta\in\Theta}$ be a family of probability densities with respect to a σ-finite reference measure μ on some measurable space Ω indexed by some open set $\Theta \subset \mathbb{R}^n, n \in \mathbb{N}$, and denote by $\mathbb{P}^\theta$ the corresponding probability measures on Ω. Let $f : \Omega \to \mathbb{R}$ be a measurable function. Then, if the family of functions $x \mapsto p^\theta(x)f(x)$ is sufficiently regular for the Leibniz rule to hold*[9]*, we have*

$$\nabla_\theta \mathbb{E}^{\mathbb{P}^\theta}(f) = \mathbb{E}^{\mathbb{P}^\theta}\left(\nabla_\theta \ln\left(p^\theta\right) f\right), \tag{9}$$

where $\mathbb{E}^{\mathbb{P}^\theta}$ denotes the expectation with respect to $\mathbb{P}^\theta$.

Proof. Using the Leibniz rule to exchange differentiation and integration, we get

$$\begin{aligned}
\nabla_\theta \mathbb{E}^{\mathbb{P}^\theta}(f) &= \nabla_\theta \int_\Omega p^\theta(x) f(x)\, \mathrm{d}\mu(x) \\
&= \int_\Omega \nabla_\theta p^\theta(x) f(x)\, \mathrm{d}\mu(x) \\
&= \int_\Omega \frac{\nabla_\theta p^\theta(x)}{p^\theta(x)} f(x) p^\theta(x)\, \mathrm{d}\mu(x) \\
&= \int_\Omega \nabla_\theta \ln\left(p^\theta(x)\right) \mathrm{d}\mathbb{P}^\theta(x) \\
&= \mathbb{E}^{\mathbb{P}^\theta}\left(\nabla_\theta \ln\left(p^\theta\right) f\right).
\end{aligned}$$

□

To derive the REINFORCE algorithm, we use lemma A.4 applied to the family $\mathbb{P}^\theta$ of joint distributions over the R_t, S_t, A_t (this distribution depends on θ through the choice of policies π_t^θ) in order to obtain, under the assumption that $\mathbb{P}^\theta$ have densities p^θ satisfying the conditions of lemma A.4, that

$$\nabla_\theta \mathbb{E}\left(R_t\right) = \mathbb{E}\left(\nabla_\theta \ln \pi_t^\theta\left(A_t \mid S_t\right) R_t\right). \tag{10}$$

A.4.3. PPO

Having introduced basic PGMs in section A.4.2, we now describe the approach used in our article, PPO. A big problem with PGMs such as REINFORCE is that the updates introduced through gradient ascent may be numerically instable. PPO, introduced in Schulman *et al* (2017), aims to solve this problem by recasting the optimization problem.

[9] We omit a further discussion on what conditions are sufficient for this statement.

Algorithm 2. PPO-Clip natural language pseudo-code, minimally adapted from Achiam (2018).

Input: Initial policy parameters θ_0, initial value function parameters ϕ_0.
for $k = 0, 1, 2, \ldots$ **do**
  Generate a sequence of realizations of states, actions and rewards according to the policies $\pi_0^{\theta_0}, \pi_1^{\theta_0}, \ldots$.
  Compute estimates of the advantage function based on the current value network $V_t^{\phi_k}$.
  Update the policy by maximizing the PPO-Clip objective L^{Clip}, typically with stochastic gradient ascent.
  Update value network parameters ϕ_k to those ϕ_{k+1} which minimize the empirically observed expectation of $(V_t^{\phi}(S_t) - R_t)^2$, typically via gradient descent.
end for

For $t \in \mathbb{N}$, we define the *advantage function at time t* as the function $\mathfrak{A}_t$ mapping each $s \in \mathcal{S}$ and $a \in \mathcal{A}$ to

$$\mathfrak{A}_t(s,a) \overset{\text{Def.}}{=} \mathbb{E}(R_t \mid S_t = s \wedge A_t = a) - \mathbb{E}(R_t \mid S_t = s). \tag{11}$$

The advantage function gives for a state s and action a the difference in expected reward if we perform action a instead of performing a random action drawn from the policy $\pi_t(\cdot \mid S_t)$. Since the advantage function depends on the policies, we will write $\mathfrak{A}_t^{\pi}$ for the advantage policy associated of policy π where the choice of the policy is not evident from the context. We note that, for ease of exposition, we only give the Definition for a single time step. In a full implementation, one may sum up to a finite horizon or approximate the full discounted future reward, see remark A.3.

PPO, just like REINFORCE, iteratively updates a set of parameters $\theta \in \Theta$ which parameterize a family of policies π_t^{θ}. We start with an initial set of parameters θ_0 and then update through the following step.

$$\theta_{k+1} \in \operatorname{argmax}_{\theta \in \Theta} \mathbb{E}\left(L^{\text{clip}}(S_t, A_t, \theta_k, \theta)\right), \tag{12}$$

where

$$L^{\text{clip}}(s,a,\theta_k,\theta) \overset{\text{Def.}}{=} \min\left(r(\theta,\theta_k)\mathfrak{A}^{\pi_t^{\theta_k}}(s,a), \min(1+\varepsilon, \max(1-\varepsilon, r(\theta,\theta_k,a,s)))\mathfrak{A}^{\pi_t^{\theta_k}}(s,a)\right) \tag{13}$$

with

$$r(\theta,\theta_k) \overset{\text{Def.}}{=} \frac{\mathrm{d}\pi_t^{\theta}(\cdot \mid s)}{\mathrm{d}\pi_t^{\theta_k}(\cdot \mid s)}(a). \tag{14}$$

We assume here implicitly that $\pi_t^{\theta}(\cdot \mid s)$ is absolutely continuous with respect to $\pi_t^{\theta_k}(\cdot \mid s)$.

A main question for PPO is how to access the advantage function. This is because the expectations in (11) are not directly accessible. Instead, we estimate it using a *value network*, whose task it is to approximate the state value function. The policy updates then use values provided by the value network. In algorithm 2, we denote the value network with parameters ϕ by V_t^{ϕ}.

Therefore, PPO is an *actor-critic* approach. In the actor-critic approach, two functions get trained concurrently. The first function, the so-called *critic*, approximates the state value function. The second function, the so-called *actor*, is the policy of the agent aiming to maximize the cumulative discounted expected reward. The actor uses values generated by the critic in order to update its parameters.

A.5. PPO hyperparameter settings

The PPO algorithm hyperparameters used in this work are summarized in table 6. For the synthetic data studies, a single baseline configuration was used for all experiments except where explicitly noted. The configuration for the real-data LHCb study required further empirical tuning as described in section 3.1.5.

Table 6. PPO hyperparameter settings for the different experimental studies.

Parameter	Symbol	Value (Synthetic Studies)	Value (LHCb Study)
Actor/critic learning rate	α	1×10^{-4}	1×10^{-4}
Optimizer	—	Adam	Adam
PPO clipping Epsilon	ϵ	0.2	0.2
Discount factor	γ	0.99	0.99
Value loss coefficient	c_1	0.5	0.5
Entropy coefficient	c_2	0.5	0.7
Update steps per episode	—	10	30

References

Aad G *et al* 2008 The ATLAS experiment at the CERN large hadron collider *JINST* **3** S08003

Aaij R *et al* 2024 The LHCb Upgrade I *JINST* **19** 05065

Aamodt K *et al* 2008 The ALICE experiment at the CERN LHC *JINST* **3** S08002

Abadi M *et al* 2016 Tensorflow: large-scale machine learning on heterogeneous distributed systems *CoRR* (arXiv:1603.04467)

Abadjiev D *et al* 2024 Autoencoder-based anomaly detection system for online data quality monitoring of the CMS electromagnetic calorimeter *Comput. Softw. Big Sci.* **8** 11

Achiam J 2018 Proximal policy optimization. OpenAI spinning up (available at: https://spinningup.openai.com/en/latest/algorithms/ppo.html) (Accessed 17 May 2024)

Adinolfi M, Archilli F, Baldini W, Baranov A, Derkach D, Panin A, Pearce A and Ustyuzhanin A 2017a LHCb data quality monitoring *J. Phys.: Conf. Ser.* **898** 092027

Alves A A *et al* 2008 The LHCb Detector at the LHC *JINST* **3** S08005

Arshad K, Ali R F, Muneer A, Aziz I A, Naseer S, Khan N S and Taib S M 2022 Deep reinforcement learning for anomaly detection: a systematic review *IEEE Access* **10** 124017–35

Asres M W *et al* 2023 Spatio-temporal anomaly detection with graph networks for data quality monitoring of the hadron calorimeter *Sensors* **23** 9679

Barhate N 2018 Minimal pytorch implementation of proximal policy optimization (available at: https://github.com/nikhilbarhate99/PPO-PyTorch)

Brockman G, Cheung V, Pettersson L, Schneider J, Schulman J, Tang J, and Zaremba W 2016 Openai gym *CoRR* (arXiv:1606.01540)

Chatrchyan S *et al* 2008 The CMS Experiment at the CERN LHC *JINST* **3** S08004

Clarke Peter *et al* LHCb collaboration 2013 *LHCb External Data Access Policy.* CERN Open Data Portal (https://doi.org/10.7483/OPENDATA.LHCb.HKJW.TWSZ)

Deja K R 2019 Using machine learning techniques for data quality monitoring in CMS and ALICE experiments *PoS* **LHCP2019** 236

Hanten J, Arnold M, Birkhan J, Caliari C, Pietralla N and Steinhorst M 2019 Enhancement of the S-DALINAC control system with machine learning methods *8th Int. Beam Instrumentation Conf.* p WEBO04

Hirlaender S and Bruchon N 2020 Model-free and Bayesian ensembling model-based deep reinforcement learning for particle accelerator control demonstrated on the FERMI FEL (arXiv:2012.09737)

Hirlaender S, Lamminger L, Zevi Della Porta G and Kain V 2023 Ultra fast reinforcement learning demonstrated at CERN AWAKE *JACoW* vol IPAC2023 p THL038

Kafkes D and Schram M 2021 Developing robust digital twins and reinforcement learning for accelerator control systems at the fermilab booster *12th Int. Particle Accelerator Conf.*

Kain V, Hirlander S, Goddard B, Velotti F M, Zevi Della Porta G, Bruchon N and Valentino G 2020 Sample-efficient reinforcement learning for CERN accelerator control *Phys. Rev. Accel. Beams* **23** 124801

Kaufmann T, Weng P, Bengs V and Hüllermeier E 2023 A survey of reinforcement learning from human feedback (arXiv:2312.14925)

Kingma D P and Ba J 2014 Adam: a method for stochastic optimization (arXiv:1412.6980)

Natarajan N, Dhillon I S, Ravikumar P and Tewari A 2013 Learning with noisy labels *NIPS*, ed C J C Burges, L Bottou, Z Ghahramani and K Q Weinberger pp 1196–204

Olivia Jullian Parra J G P 2025 Human in the loop rl for DQM (available at: https://doi.org/10.5281/zenodo.17019894)

Pang X, Thulasidasan S and Rybarcyk L 2020 Autonomous control of a particle accelerator using deep reinforcement learning (arXiv:2010.08141)

Paszke A *et al* 2019 Pytorch: an imperative style, high-performance deep learning library *CoRR* (arXiv:1912.01703)

Pol A A, Azzolini V, Cerminara G, De Guio F, Franzoni G, Pierini M, Široký F and Vlimant J R 2019a Anomaly detection using deep autoencoders for the assessment of the quality of the data acquired by the CMS experiment *Technical Report* CERN (available at: https://cds.cern.ch/record/2650715)

Pol A A, Berger V, Cerminara G, Germain C and Pierini M 2019b Trigger rate anomaly detection with conditional variational autoencoders at the CMS experiment *Machine Learning and the Physical Sciences Workshop at the 33rd Conf. on Neural Information Processing Systems (NeurIPS)* (available at: https://inria.hal.science/hal-02428005)

Pol A A, Cerminara G, Germain C and Pierini M 2022 *Data Quality Monitoring Anomaly Detection* (World Scientific Publishing Company) Ch 5, pp 115–49

Pol A A, Cerminara G, Germain C, Pierini M and Seth A 2019c Detector monitoring with artificial neural networks at the CMS experiment at the CERN Large Hadron Collider *Comput. Softw. Big Sci.* **3** 3

Roberts S W 1959 Control chart tests based on geometric moving averages *Technometrics* **1** 239–50

Schulman J, Wolski F, Dhariwal P, Radford A and Klimov O 2017 Proximal policy optimization algorithms (arXiv:1707.06347)

Shorten C and Khoshgoftaar T M 2019 A survey on image data augmentation for deep learning *J. Big Data* **6** 1–48

Su C, Wang Z, Chen X, Jia Y, Qi X and Jia D 2023 Reinforcement control for LEBT and RFQ of linear accelerators *JACoW* vol IPAC2023 p WEA099

Wen Q, Sun L, Yang F, Song X, Gao J, Wang X and Xu H 2021 Time series data augmentation for deep learning: a survey *Proc. 13th Int. Joint Conf. on Artificial Intelligence, IJCAI-21*, ed Z-H Zhou (International Joint Conferences on Artificial Intelligence Organization) pp 4653–60

Williams R J 1992 Simple statistical gradient-following algorithms for connectionist reinforcement learning *Mach. Learn.* **8** 229–56

Xu C *et al* 2023 Beyond PID controllers: PPO with neuralized PID policy for proton beam intensity control in Mu_2e *37th Conf. on Neural Information Processing Systems*

Zurbano Fernandez I et al 2020 High-Luminosity Large Hadron Collider (HL-LHC): technical design report *10/2020*